

 BÁO CÁO BÀI TẬP LỚN · LẬP TRÌNH NÂNG CAO (LTNC) · 2025–2026

Online Auction System

Hệ thống Đấu giá Trực tuyến Real-time

 MÔN HỌC

Lập trình Nâng cao

 NĂM HỌC

2025 – 2026

 KIẾN TRÚC

Client-Server + MVC

 DESIGN PATTERNS

10 patterns áp dụng

 Java 25

 JavaFX 21

 MySQL 8

 TCP Socket

 Maven Multi-Module

 JUnit 5

 GitHub Actions CI/CD

 Video Demo: youtube.com/watch?v=qxf3Nleffd0

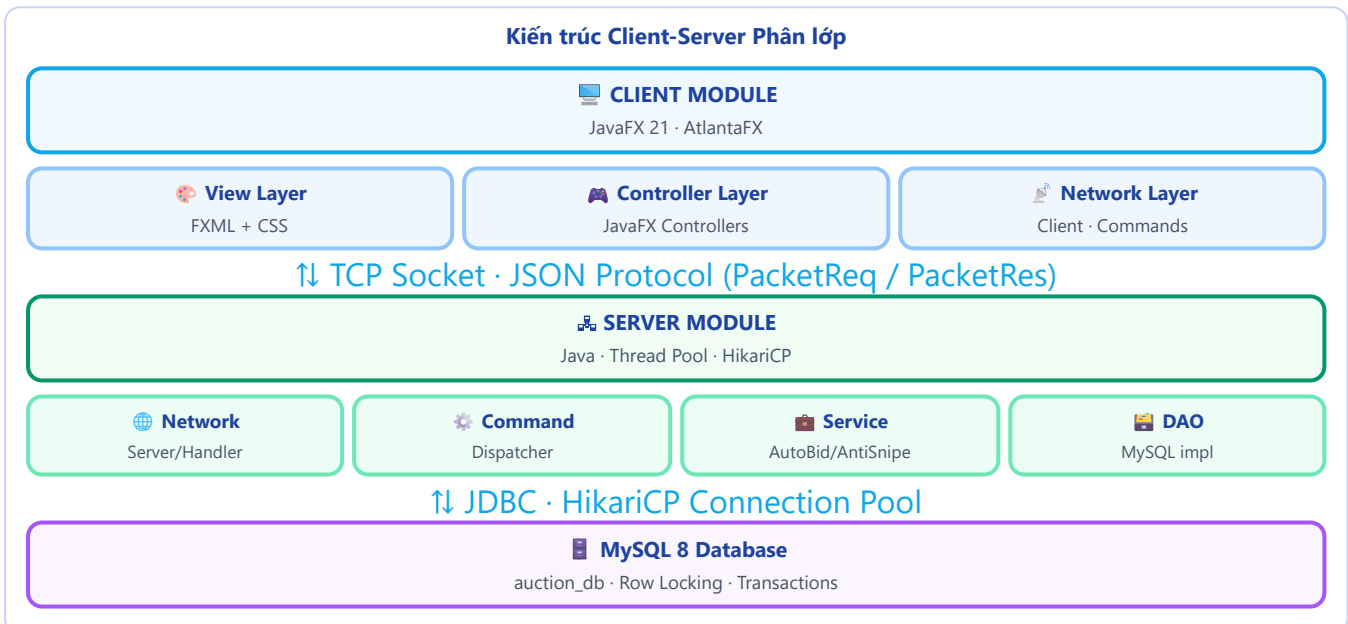
 GitHub: github.com/giabaotq07/AuctionOnlineSystem

1 Mục tiêu & Phạm vi thực hiện

Bài toán: Xây dựng sàn đấu giá trực tuyến nhiều người dùng đồng thời, hỗ trợ đặt giá theo thời gian thực qua kết nối TCP Socket, với đầy đủ nghiệp vụ phân quyền, ví điện tử, tự động đấu giá, chống snipe và chat trong phòng. Dự án áp dụng nghiêm ngặt OOP và 10 Design Patterns kinh điển trong Java.

Tiêu chí	Mô tả
Loại ứng dụng	Desktop Client-Server, đa người dùng đồng thời
Giao tiếp	TCP Socket, giao thức JSON tùy chỉnh (PacketReq / PacketRes với Gson)
Phân quyền	3 vai trò: Admin Seller Bidder
Cơ sở dữ liệu	MySQL 8 + HikariCP Connection Pool + Row Locking (SELECT ... FOR UPDATE)
Giao diện	JavaFX 21 + AtlantaFX (FXML + CSS), biểu đồ Canvas thời gian thực
Build	Maven Multi-Module: common · server · client

2 Kiến trúc tổng thể hệ thống



3 Thiết kế hướng đối tượng (OOP)

• **Trừu tượng (Abstraction):** Định nghĩa các hành vi chung qua các lớp trừu tượng và giao diện như `Entity`, `Item`, interface `Command`, `UserDAO`, `AuctionDAO`.

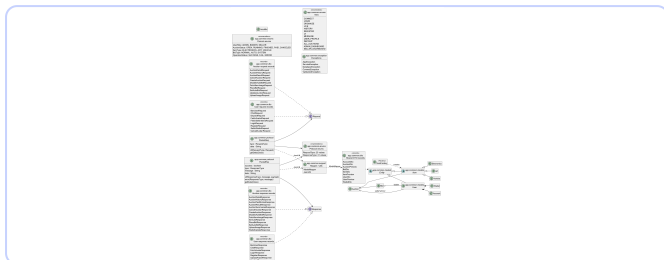
• **Đóng gói (Encapsulation):** Các thuộc tính của `User`, `Wallet`, `Auction` ... đều đặt ở `private/protected`, truy xuất an toàn qua getter/setter và bọc logic xử lý nội bộ.

• **Kế thừa (Inheritance):** Tổ chức cây kế thừa phân cấp: `Entity` → `Item` → `Electronics`, `Art`, `Vehicle`; hoặc `Command` → `SafeCommand` → 25 lệnh cụ thể.

• **Đa hình (Polymorphism):** Ghi đè phương thức `getItemType()` ở các lớp con sản phẩm; các lớp lệnh cụ thể ghi đè hàm `execute()` để tự quyết định logic nghiệp vụ.

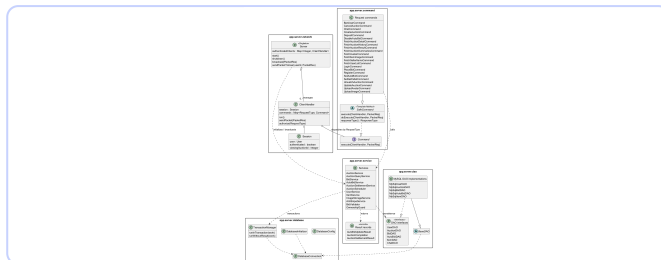
3 Sơ đồ UML & Mô tả kiến trúc chi tiết

3.1 Sơ đồ UML – Gói Common (Mô hình dữ liệu)



Kế thừa `Entity` → `Item` → `Electronics/Art/Vehicle` ; thực thể `User` , `Wallet` , `Auction` , `Bid` , `AutoBid` ; giao thức `PacketReq/Res` .

3.2 Sơ đồ UML – Gói Server (Nghệ vụ & CSDL)

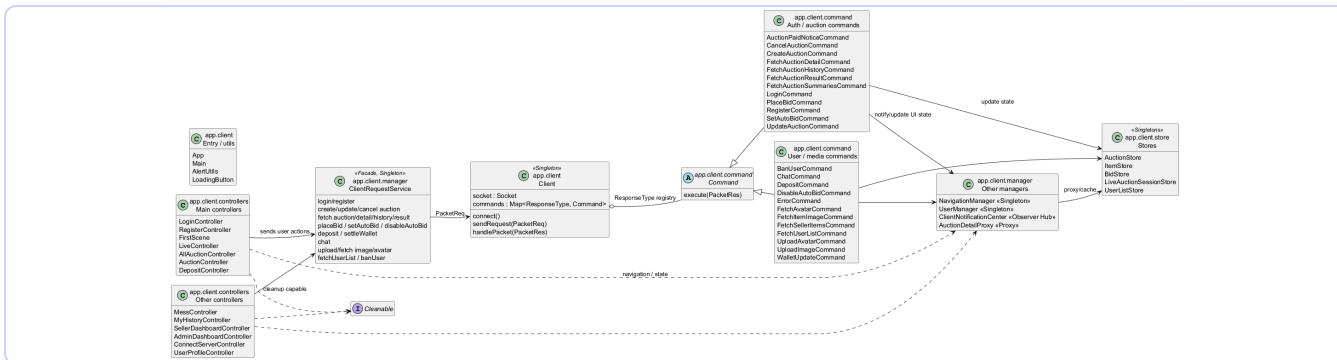


Lớp `Server` kết nối, `ClientHandler` giữ session, `Command` pattern định tuyến, các Service nghiệp vụ (`AutoBid`, `AntiSnipe`, `Scheduler`), `DAO` và `TransactionManager` .

4 Mô tả các lớp/thành phần quan trọng theo kiến trúc

Thành phần	Lớp đại diện chính	Chức năng & Vai trò chi tiết trong kiến trúc hệ thống
Mô hình dữ liệu (Domain Models)	<code>User</code> , <code>Item</code> (abstract) → <code>Electronics</code> , <code>Art</code> , <code>Vehicle</code> , <code>Auction</code> , <code>Bid</code> , <code>Wallet</code>	Đóng gói các đối tượng dữ liệu lõi của hệ thống. <code>User</code> quản lý thông tin định danh và vai trò. <code>Item</code> phân cấp kế thừa các thuộc tính sản phẩm đặc thù. <code>Auction</code> quản lý trạng thái, giá cả, thời gian đóng/mở phiên đấu giá. <code>Bid</code> lưu vết chi tiết từng lượt đặt giá.
Giao thức truyền tin (Protocol DTOs)	<code>PacketReq</code> , <code>PacketRes</code> (DTO record), <code>Packet</code> (abstract)	Định nghĩa cấu trúc dữ liệu JSON để giao tiếp qua mạng. Đóng gói thông điệp gửi đi (<code>PacketReq</code>) gồm loại yêu cầu (<code>RequestType</code>) và payload, thông điệp phản hồi (<code>PacketRes</code>) gồm mã lỗi, thông điệp và kết quả. Tối ưu bằng thông bằng cách loại bỏ liên kết thừa.
Mạng & Kết nối (Socket Server)	<code>Server</code> (Singleton), <code>ClientHandler</code> , <code>SessionManager</code>	<code>Server</code> lắng nghe TCP cổng <code>12345</code> , khởi tạo Thread Pool quản lý đa kết nối song song. <code>ClientHandler</code> duy trì luồng kết nối riêng biệt cho mỗi Client để tiếp nhận gói tin. <code>SessionManager</code> ánh xạ kết nối sang ID người dùng đã đăng nhập để thực hiện gửi thông điệp đích danh hoặc broadcast.
Định tuyến nghiệp vụ (Command Registry)	<code>Command</code> (interface), <code>ServerCommandRegistry</code> , các lớp <code>Command</code> nghiệp vụ	Áp dụng Command Pattern. <code>ServerCommandRegistry</code> ánh xạ <code>RequestType</code> đến lớp xử lý tương ứng (như <code>LoginCommand</code> , <code>PlaceBidCommand</code>). Khi nhận request, Server gọi phương thức <code>execute()</code> của command đó để thực thi nghiệp vụ, tránh cấu trúc rẽ nhánh rườm rà.
Dịch vụ nghiệp vụ (Business Services)	<code>UserService</code> , <code>AuctionService</code> , <code>BidService</code> , <code>ItemService</code>	Đóng gói quy tắc nghiệp vụ chính của hệ thống. <code>UserService</code> xác thực/đăng ký người dùng; <code>AuctionService</code> quản lý tạo/tham gia phòng đấu giá; <code>BidService</code> điều phối đặt giá và cập nhật vị; <code>ItemService</code> xử lý lưu trữ/quản lý sản phẩm.
Dịch vụ hệ thống (System Services)	<code>AutoBidService</code> , <code>AntiSnipeService</code> , <code>AuctionScheduler</code>	<code>AutoBidService</code> chạy ngầm để đặt giá tự động dựa trên bước tăng và mức giá trần cấu hình. <code>AntiSnipeService</code> tự động cộng thêm 60 giây vào thời gian kết thúc khi phát hiện lượt bid ở 30 giây cuối. <code>AuctionScheduler</code> quét CSDL chu kỳ 1s để đổi trạng thái phiên và thanh toán tiền.
Tầng truy xuất CSDL (Persistence Layer)	<code>UserDAO</code> , <code>AuctionDAO</code> , <code>BidDAO</code> , <code>ItemDAO</code> (MySQL JDBC)	Áp dụng Data Access Object (DAO) Pattern để cô lập mã SQL khỏi logic nghiệp vụ. Sử dụng JDBC kết hợp Connection Pool HikariCP hiệu năng cao. Các lớp DAO thực thi các câu lệnh truy vấn CRUD và ánh xạ kết quả CSDL thành các đối tượng model Java.
Giao dịch & Đồng thời (Concurrency Logic)	<code>TransactionManager</code> , Row-Locking (<code>SELECT ... FOR UPDATE</code>)	<code>TransactionManager</code> điều khiển commit/rollback thủ công, đảm bảo tính nguyên tử (Atomicity). Sử dụng cơ chế khóa dòng ở CSDL khi thực hiện đặt giá, đảm bảo tại một thời điểm chỉ một luồng được cập nhật giá trị phiên đấu giá, triệt tiêu race condition.

5



Lớp **Client** duy trì Socket, các Manager (**NavigationManager** đổi màn hình, **ClientRequestService** Facade, **UserManager** lưu tài khoản), các Store lưu trạng thái và hệ thống 13 JavaFX Controllers.

6

① Singleton

Áp dụng: DatabaseConnection, Server, Client, NavigationManager, UserManager, AuctionStore.

Lý do: Đảm bảo duy nhất một kết nối DB/Socket trong hệ thống; đồng bộ hóa trạng thái ứng dụng.

③ Command

Áp dụng: Server: `LoginCommand` , `PlaceBidCommand` ... Client: `UpdateAuctionCommand` , `WalletUpdateCommand` ...

Lý do: Đóng gói các yêu cầu mạng thành đối tượng xử lý riêng biệt; tránh cấu trúc if-else/switch lồng nhau.

⑤ DAO (Data Access Object)

Áp dụng: UserDAO , AuctionDAO , BidDAO ... định nghĩa giao diện truy xuất MySQL database.

Lý do: Tách biệt hoàn toàn tầng logic nghiệp vụ với tầng truy vấn dữ liệu SQL; để bảo trì và chuyển đổi CSDL.

⑦ Proxy

Áp dụng: `AuctionDetailProxy` bọc quanh thông tin chi tiết phiên đấu giá của Client.

Lý do: Lazy loading thông tin chi tiết; lưu cache tạm thời giúp tối ưu mạng và tránh tải lại dữ liệu liên tục.

⑨ Registry

Áp dụng: `ClientHandler` dùng `EnumMap` để lưu trữ và tra cứu nhanh các lệnh.

Lý do: Ánh xạ trực tiếp $O(1)$ từ loại gói tin sang lớp xử lý; cấu trúc đăng ký lệnh tập trung, trực quan.

② Factory Method

Áp dụng: `ItemFactory` khởi tạo `Electronics/Art/Vehicle` ; static
factory tạo `PacketReg/Res` .

Lý do: Che giấu chi tiết khởi tạo đối tượng con; dễ dàng mở rộng thêm các danh mục sản phẩm mới.

④ Observer (Pub-Sub)

Áp dụng: Giao diện đăng ký lắng nghe packet qua `PacketListener` ; Client nhận tin và phát tán sự kiện.

Lý do: Cập nhật giao diện theo thời gian thực (realtime) mà không cần Polling; giảm phụ thuộc giữa UI và Network.

⑥ DTO (Data Transfer Object)

Áp dụng: `LoginRequest`, `PlaceBidRequest`, `AuctionPreview` ... dùng trao đổi dữ liệu qua mạng.

Lý do: Tối ưu hóa bằng thông bằng cách chỉ truyền tải các trường dữ liệu cần thiết thay vì truyền tải cả Entity.

⑧ MVC

Áp dụng: View (FXML/CSS) · Controllers ([LiveController](#) ...) · Models ([User](#) , [Auction](#) ...) dùng JavaFX.

Lý do: Phân định rõ ràng trách nhiệm hiển thị giao diện, kiểm soát và lưu trữ dữ liệu; tăng khả năng tái sử dụng.

⑩ Facade

Áp dụng: `ClientRequestService` bọc toàn bộ logic tạo và gửi packet TCP Socket của Client.

Lý do: Đơn giản hóa các lời gọi hàm từ giao diện JavaFX; giấu chi tiết kết nối mang phức tạp phía sau.

7 Chức năng đạt được theo Barem điểm & Hướng giải quyết

Chức năng	Barem	Hướng giải quyết (Giải pháp công nghệ trong dự án)	Lý do lựa chọn giải pháp
Đăng ký & Đăng nhập	Cơ bản	Bảo mật khẩu bằng thư viện BCrypt ở Server. Quản lý trạng thái phiên làm việc (user, role) tại ClientHandler kết nối Socket.	Bảo mật dữ liệu mật khẩu người dùng; giữ session tại kết nối giúp Server phân quyền và định tuyến truyền tin realtime chính xác.
Quản lý sản phẩm	Cơ bản	Phân cấp sản phẩm (Electronics/Art/Vehicle) kế thừa Item. Tạo qua ItemFactory. Quản lý tệp ảnh qua ImageStorageService.	Dễ mở rộng thêm danh mục sản phẩm mà không đổi mã nguồn cốt lõi (OCP); lưu tệp ảnh trên ổ đĩa giúp giảm tải cho DB.
Tạo & Lập lịch đấu giá	Cơ bản	Cấu hình thời gian bắt đầu/kết thúc. AuctionScheduler sử dụng ScheduledExecutorService để tự động chạy luồng kích hoạt/đóng phiên.	Tự động hóa hoàn toàn chu kỳ phiên đấu giá; chạy nền độc lập giúp luồng chính nhận kết nối Socket của Server không bị tắc nghẽn.
Đặt giá Real-time	Cơ bản	Giao tiếp TCP Socket dùng DTO PacketReq/PacketRes. Khi có giá mới, Server lập tức broadcast gói tin đến tất cả Client đang xem phòng.	Độ trễ truyền tin cực thấp (vài ms), giảm thiểu băng thông so với HTTP Polling; giao diện UI JavaFX cập nhật giá tức thì.
Ví điện tử & Đặt cọc	Cơ bản	Lưu trữ số dư khả dụng và số tiền đóng băng (frozenAmount). Khi tham gia phòng, hệ thống tự động đóng băng cọc và hoàn trả khi bị vượt giá.	Đảm bảo khả năng thanh toán của Bidder, tránh tình trạng quyt nợ; tự động hoàn cọc nhanh nâng cao trải nghiệm người dùng.
Concurrency Control	Cơ bản	Quản lý giao dịch qua TransactionManager, khóa dòng dữ liệu tại CSDL bằng câu lệnh SELECT ... FOR UPDATE khi đặt cọc và đặt giá.	Ngăn chặn race conditions khi nhiều người đặt giá cùng lúc; đảm bảo tính toàn vẹn và nhất quán tuyệt đối (ACID) của giao dịch tiền tệ.
Chat trong phòng	Cơ bản	Command ChatCommand nhận tin nhắn và broadcast đến danh sách các Client có thuộc tính watchingAuction tương ứng trên Server.	Chỉ truyền tin nhắn cho các Client thực sự đang ở trong phòng đấu giá đó, giúp giảm tối đa tải cho mạng và tài nguyên xử lý.
Thông báo & Lịch sử	Cơ bản	Server tự động gửi gói tin thông báo WALLET_UPDATED/CHAT_MESSAGE khi bị vượt giá. Lịch sử đặt giá được truy vấn động qua BidDAO.	Thông báo ngay để Bidder kịp thời đưa ra quyết định đặt giá tiếp theo; đảm bảo tính công khai và minh bạch lịch sử đấu giá.
Đấu giá tự động	Nâng cao	Người dùng đặt giá trần và bước tăng. AutoBidService đóng băng số tiền bằng giá trần. Khi bị vượt giá, Server tự động bid hộ theo bước tăng.	Hỗ trợ tối đa người dùng bận rộn; việc đóng băng cọc trần trước giúp đảm bảo tính hợp lệ về năng lực tài chính trước khi tự động bid.
Chống giật giải phút chót	Nâng cao	AntiSnipeService tự động cộng thêm 60 giây vào thời gian kết thúc phiên đấu giá nếu phát hiện bid hợp lệ ở 30 giây cuối (tối đa 5 lần).	Loại bỏ hành vi đầu cơ giây cuối (sniping); tối đa hóa giá trị sản phẩm bán ra; tạo sự cạnh tranh lành mạnh và sòng phẳng.
Biểu đồ biến động giá	Nâng cao	Tự động vẽ đồ thị dạng đường (Line Chart) thời gian thực bằng JavaFX Canvas GraphicsContext2D dựa trên dữ liệu lịch sử đặt giá nhận được.	Trực quan hóa xu hướng giá của phiên; vẽ trực tiếp lên Canvas đem lại hiệu năng xử lý đồ họa mượt mà hơn nhiều Chart mặc định.
Cấu hình IP linh hoạt	Nâng cao	Đọc địa chỉ IP Server từ file properties bên ngoài JAR, hoặc cho phép người dùng thay đổi trực tiếp tại màn hình kết nối của GUI.	Dễ dàng triển khai, di chuyển chạy Server và Client trên các máy tính khác nhau trong mạng LAN/Internet mà không cần build lại ứng dụng.

8 Tích hợp hệ thống, Chất lượng mã & Kiểm thử

• **Unit Test (JUnit 5):** Triển khai hơn 30 tệp kiểm thử tự động (như `UserServiceTest`, `AutoBidServiceTest` ...) phủ toàn diện logic Service, Command và DAO để đảm bảo độ tin cậy tuyệt đối.

• **Tự động hóa CI/CD:** Cấu hình GitHub Actions (tệp `maven.yml`) tự động chạy khi push/PR lên nhánh `main`. Sử dụng `xfx3b-run` để giả lập màn hình đồ họa kiểm thử JavaFX.

• **Quản lý mã nguồn & Convention:** Quản lý đa module bằng Maven; tuân thủ Google Java Style Guide; tổ chức package phân tầng MVC chuẩn hóa; lịch sử commit rõ ràng và minh bạch.

Báo cáo PDF

Báo cáo nộp chính thức · Đúng 5 trang

Video Demo

youtube.com/watch?v=qxf3Nleffd0

GitHub Repository

github.com/giabaotq07/AuctionOnlineSystem